NURSESYNC

Product Requirements Document: Agent Build

Agent name: NurseSync, a team of 4 AI agents for hospital nurse scheduling

Owner(s): Cheich Toure & Kerrian Gordon

Date: October 21, 2026

How to read this document

Sections 1 and 2 describe the full product. Section 3 describes the complete design across all three phases. Each subsection is labelled so a reader knows which phase it belongs to: [Phase 1 — this week] means build and test it now. [Phase 2] means it is designed but not built yet. [Phase 3] means it comes after phase 2 passes. Do not implement a phase 2 or phase 3 section until its phase starts. The labels are the source of truth for scope.

1. PROBLEM

Nurse managers at hospitals spend hours every week building schedules by hand, because staffing needs change daily and nurse availability, time-off requests and staffing rules live in different places. The result is understaffed shifts, extra overtime, rule violations that are caught late, and tired nurses.

Supporting Context (optional)

- A manager must balance patient demand, each nurse's availability and approved time off, and rules like rest between shifts, overtime limits and the right skills on each unit, all at the same time.
- Call-outs and swap requests arrive after the schedule is posted, so the manager has to rebuild part of it quickly.
- Schedules built by hand are hard to audit, so rule violations are often found only after the schedule is posted.

1a. Opportunity

Give every nurse manager a draft schedule that already follows the rules, so they review and approve it instead of building it. This returns hours each week to patient care and cuts down on last-minute gaps and overtime.

Size of the Opportunity

- A single weekly schedule for one hospital unit can take a manager hours to build and check by hand. This is an assumption to validate with nurse managers or a published source.
- Every hour saved per manager each week goes back to patient care and staff support, and fewer rule violations means less overtime cost and less risk.

1b. Users & Needs

Primary user(s): Nurse managers and charge nurses who build the weekly schedule for one hospital unit. They care most about full coverage, fairness and no rule violations.

Secondary users: Staff nurses, who enter availability and request time off, and hospital administrators, who care about overtime cost and compliance.

Key User Needs

As a nurse manager, I need a draft schedule that already follows the rules because checking every rule by hand takes too long.

As a nurse manager, I need to see why the schedule was built this way because I have to trust it before I approve it.

As a staff nurse, I need my availability and approved time off respected because I plan my life around my shifts.

As a staff nurse, I need swap and call-out changes handled quickly and fairly because I need predictable time away from work. Swaps are planned for a later version.

2. PROPOSED SOLUTION

NurseSync is a team of four AI agents for hospital nurse scheduling. It runs every week, and again whenever a nurse calls out. A Demand agent estimates how many nurses each day needs, an Availability agent collects who can work and who asked for time off, a Scheduler agent builds a draft, and a Rules Checker agent reviews the draft and sends it back for fixes, up to 3 times, until it follows the rules. The nurse manager then reviews and approves the schedule. The agents inform the human, and the human makes the call. Version one covers one unit and one week, including mid-week call-outs. Shift swaps between nurses are out of scope for version one.

2a. Value Proposition

Nurse managers who struggle to build compliant schedules, because demand, availability and staffing rules are scattered, use NurseSync, a team of AI agents that drafts and checks the weekly schedule. Unlike building it by hand in a spreadsheet, it checks every rule before the manager sees the draft, helping them approve a schedule in minutes instead of building one for hours.

2b. Top 3 MVP Value Props

The Vitamin *(must-have baseline)*: Every shift in the week is covered by an available, qualified nurse, with no one scheduled outside their availability or approved time off.

The Painkiller *(solves the core pain)*: No more building the schedule by hand. The manager gets a full draft that has already passed every rule check.

The Steroid *(the magic moment)*: When a nurse calls out, NurseSync suggests replacements in minutes, with the reason for each, ready for the manager to approve.

2c. Success Metrics

Goal	Signal	Metric	Target
Save manager time	Managers stop building schedules by hand	Minutes for NurseSync to produce a passing draft and check-and-fix rounds used	Under 5 minutes and 3 rounds or fewer (placeholder until the first run)
Full coverage	Every shift has the nurses it needs	Percent of shifts fully staffed in the approved schedule	At least 95%

Trustworthy drafts (quality)	Drafts follow every rule	Hard-rule violations in drafts shown to the manager	0
Fair schedules	Night and weekend shifts are spread evenly	Gap between the most and fewest night and weekend shifts across nurses	At most 2
Multi-agent value	The agent team beats one agent doing everything	Hard-rule violations and fully staffed shifts, compared with a single-agent baseline on the same synthetic data	Fewer violations at equal or better coverage

3. AGENT REQUIREMENTS

3a. Tools — [Phase 1: get_roster, check_rules] [Phase 2: get_demand, get_availability] [Phase 3: get_callouts]

Tool name	What it does	API it calls	Data it returns
get_demand	Estimates how many nurses each unit needs per shift (Demand agent)	Shift Board data: sample patient counts per unit	Unit, date, shift, patient count, nurses needed
get_availability	Reads who can work when and who asked for time off (Availability agent)	Shift Board data: availability and leave	Nurse, date, shift, available, reason (none, approved time off or call-out)
get_roster	Reads each nurse's unit, skills and hours worked (Scheduler agent)	Shift Board data: roster	Name, unit, certifications, hours this week
check_rules	Tests a draft against rest-time, overtime and skill-mix rules, using the roster and approved time off (Rules Checker agent)	Rules table in our own code	Pass or fail, and each violation found
get_callouts	Reads call-outs reported after the schedule is posted (Availability agent)	Shift Board data: call-outs	Nurse, date, shift, time reported

3b. System Prompts v0 (one per agent) — [Phase 1: Scheduler, Rules Checker] [Phase 2: Demand, Availability, Orchestrator] [Phase 3: call-out handling]

NurseSync has four agents and a small piece of plain code that runs them. Each agent has its own prompt, so each one has one job and can be tested on its own.

Rules for every agent

Never invent data. Never send anything to nurses; the manager approves and decides what happens next. If data is missing for a nurse or a day, list it under "Needs manual review" instead of guessing.

Orchestrator (plain code, not an agent) — [Phase 2 and Phase 3]

Runs every week and whenever a call-out is reported.

1. Take a snapshot of all Shift Board data (roster, availability, call-outs, patient counts and the rules table) and assign a snapshot_id. Every tool in this run reads from that snapshot.

2. Run the Demand agent and the Availability agent at the same time. If an agent times out, retry it once before stopping.

3. Check both results in code. Write the status field for each row: "missing" when the data is absent or the tool returned an error, "needs_review" when the data is present but a check flags it, and "ok" otherwise. One exception: if approved leave from Shift Board covers a day but the Availability agent says the nurse is available, the orchestrator sets the row to unavailable (available false, reason approved_time_off), marks it "needs_review", and adds a note saying it was overridden, because check_rules reads only this row. An agent cannot mark its own row "ok". Reject any result that does not parse or that has a value outside the allowed lists. If any required row is missing, stop and tell the manager what is missing before the Scheduler runs.

4. Cross-check that every nurse_id in the availability rows appears in the roster. Add a "missing" needs_manual_review row for any nurse in availability with no roster entry.

5. Give the checked demand and availability results to the Scheduler agent to build a draft.

6. Run check_rules on the draft in code. If it finds hard violations, give them to the Rules Checker agent to add a fix_hint to each, compare the agent's list to the code's list (same rule, nurse_id, date and shift), and use the code's list if they differ. Send the violations back to the Scheduler and check again, up to 3 times. A missing or unparseable result counts as a failure, never as a pass.

7. Show the manager the passing draft, or after 3 tries the best draft with its open violations. The approval view lists open gaps, items under "Needs manual review", unresolved issues and soft tradeoffs first, then the full schedule.

Manager actions: the manager can approve the draft, change a demand number (the system starts a new case with the updated demand, keeps the existing availability result, reruns from the Scheduler step only, and marks the old case out_of_date), or reject it. Reject ends the case with the outcome "rejected", and the manager builds that week by hand. Sending the approved schedule to nurses is out of scope for version one; the manager posts it themselves.

Escalation: If the draft still fails the rules after 3 tries, stop and hand it to the manager with the open violations.

Late call-outs: A call-out reported after the Availability agent has run starts a new run on a new snapshot for the affected shifts only. The new run keeps the rest of the week's assignments from the earlier draft as locked assignments, and the Scheduler changes only the affected shifts. check_rules still checks the whole week, because a replacement nurse's other shifts affect rest time and weekly hours. The new case record stores the earlier case's id as parent_case_id. If a draft is already waiting for approval, mark its case record as out_of_date and tell the manager. The manager cannot approve an out_of_date record. If the manager changes a demand number before approving, the current case is marked out_of_date and a new case record starts with its own case_id and snapshot_id. The new record stores the old case's id as parent_case_id, uses the updated demand and the existing availability result, and reruns from the Scheduler step only. The old record keeps the manager's change in decision_details and points to the new case with newer_case_id. History stays one case per run.

Handoff format (Demand and Availability to Scheduler) — [Phase 2]

(Phase 1 reads demand and availability directly from files in the same format. The Scheduler and Rules Checker use these fields in phase 1; the orchestrator validation and status-writing rules apply from phase 2 onward.)

Each result is a list of rows. Every field is required unless noted. The agent writes the data fields. The orchestrator code writes the status field after its checks.

Demand row:

- unit (text)
- date (YYYY-MM-DD)
- shift ("day" or "night")
- patient_count (whole number; the number the agent used; empty when status is "missing")
- nurses_needed (whole number, 0 or more; empty when status is "missing")
- status ("ok", "needs_review", or "missing") — written by the orchestrator
- note (text; required when status is not "ok")

Availability row:

- nurse_id (text)
- date (YYYY-MM-DD)
- shift ("day" or "night")
- available (true or false; empty when status is "missing")
- reason ("none", "approved_time_off", or "call_out"; empty when status is "missing")
- status ("ok", "needs_review", or "missing") — written by the orchestrator
- note (text; required when status is not "ok")

The orchestrator flags a demand row as "needs_review" when: nurses_needed differs from patient_count divided by the ratio, rounded up; or the row is more than 2 nurses away from the median for the same shift type across the week; or the shift needs 0 nurses when others need some. A "needs_review" row is not a rejection; the Scheduler receives it and handles it as described in the Scheduler section. Change shift names to match the unit.

Phase 1 test harness — [Phase 1]

In phase 1 there is no orchestrator. A small Python script acts as the harness for testing. It:

- Reads the demand file and the availability file from disk.
- Passes them to the Scheduler agent and receives a draft.
- Runs check_rules on the draft and the input files in code. If there are hard violations, passes them to the Rules Checker agent to add fix_hints, checks the agent's list against the code's list, and uses the code's list if they differ.
- If violations exist and the round is less than 3, passes the violations back to the Scheduler and repeats from the second step.
- After 3 rounds or a passing draft, prints the final draft, open violations, unfixable rows, and needs_manual_review rows.

Cheich builds this harness. Kerrian builds the Scheduler and Rules Checker prompts. They test together using the phase 1 eval cases.

Phase 1 input files (JSON, one file per input type)

demand.json example row: { "unit": "7W", "date": "2026-10-26", "shift": "day", "patient_count": 20, "nurses_needed": 4, "status": "ok", "note": "" }

availability.json example row: { "nurse_id": "N001", "date": "2026-10-26", "shift": "day", "available": true, "reason": "none", "status": "ok", "note": "" }

roster.json example row: { "nurse_id": "N001", "name": "Ada Brown", "unit": "7W", "certifications": ["ACLS"], "hours_this_week": 0 }

rules.json: { "min_rest_hours": 10, "max_weekly_hours": 40, "max_consecutive_shifts": 3, "required_certifications_per_shift": ["ACLS"], "ratio_patients_per_nurse": 5, "max_fairness_gap": 2, "shift_times": { "day": { "start": "07:00", "end": "19:00" }, "night": { "start": "19:00", "end": "07:00" } } }. Each assignment's

shift_start and shift_end must equal shift_times for its shift. check_rules rejects any draft whose times differ, so a draft cannot shrink its own hours.

The harness passes all four files to check_rules. check_rules reads only the assignments from the draft and uses the roster and rules files directly. It never reads the Scheduler's notes, reasoning, or unfixable rows.

Draft format: Scheduler to Rules Checker and back — [Phase 1]

Each draft has:

- case_id (in phase 1, the harness generates this as a simple string like "case-1"; in phase 2 the orchestrator generates it)
- snapshot_id (in phase 1, the harness sets this to "phase1-snapshot"; in phase 2 the orchestrator generates it)
- version (1, 2, or 3 — one per fix round; the harness increments this before each Scheduler call)
- assignments (list of: unit, date, shift, nurse_id, shift_start HH:MM, shift_end HH:MM)
- open_gaps (list of: unit, date, shift, nurses_needed, nurses_assigned, gap_reason — one of "missing", "no_available_nurse", or "no_certified_nurse")
- needs_manual_review (list of: item_type — "demand_day" or "nurse", id, note)
- notes (list of one-line reasoning strings for each hard choice)
- unfixable (list of: rule, nurse_id, date, shift, reason — one row per violation the Scheduler could not fix)
- suggestions (call-outs only; list of: date, shift, replacement nurse_id, reason — one plain sentence)

check_rules reads only the assignments and the input files, never the Scheduler's notes or unfixable rows, and rejects a draft it cannot parse. This is intentional: the Rules Checker must reach the same pass or fail result regardless of how the Scheduler explained its choices. Reasoning notes are for the manager, not for the checker.

Violation format: check_rules and Rules Checker to Scheduler — [Phase 1]

check_rules returns a list of violations. Each violation has:

- rule ("rest_time", "weekly_hours", "consecutive_shifts", "certification", "availability", "coverage", or "fairness")
- severity ("hard" or "soft")
- nurse_id (empty for coverage violations)
- date (YYYY-MM-DD)
- shift ("day" or "night")
- found (what the draft has, e.g. "8 hours of rest" or "42 hours this week")
- limit (what the rule requires, e.g. "10 hours of rest" or "40 hours max")
- fix_hint (one plain sentence saying what to change — added by the Rules Checker agent)

The Scheduler fixes each hard violation by its rule, nurse_id, date and shift, in this order: coverage and availability first, then the rest by date and then shift, because moving a nurse can change the rest-time and hours checks. The Scheduler ignores soft tradeoffs while fixing. Soft violations are not sent back to the Scheduler. They are reported to the manager through the approval view and do not fail a draft.

check_rules also returns a pass or fail result with the case_id and version of the draft it checked. The orchestrator rejects any result whose case_id or version does not match the current draft, and treats a missing or unparseable result as a failure, never as a pass.

Case record (saved as one JSON file per run; Cheich owns) — [Phase 2]

Each run creates one record:

- case_id (unique, generated at start)
- snapshot_id
- start_time
- trigger ("weekly" or "call_out")
- affected_shifts (list of date + shift pairs; all shifts for a weekly run)
- parent_case_id (call-out runs and demand-change runs: the earlier case this run updates)
- status ("building", "waiting_approval", "approved", "rejected", or "out_of_date")

- demand_result (demand rows with statuses)
- availability_result (availability rows with statuses)
- drafts (list of draft versions)
- violations (violations for each version)
- rounds_used (1, 2, or 3)
- outcome ("passed", "stopped_after_3_rounds", "stopped_missing_input", or "rejected")
- manager_decision ("approved" or "rejected") and decision_time
- stop_reason (one plain sentence; set when the run stopped early, for example because required input is missing)
- newer_case_id (set when status is "out_of_date": the case that replaced this one)
- decision_details (what the manager did beyond approved or rejected: action, demand_changes, acknowledged_open_issues, new_case_id)
- callouts (call-out runs only: the nurse, date, shift and report time of each call-out)
- unusual (one-line notes from the Demand agent; advisory text for the manager, never used by code to decide anything)
- input_review (needs_manual_review items about the inputs: the agents' own items and any nurse in the availability data who is not on the roster; needs_manual_review is the draft's items plus these, without repeats)

A late call-out or a manager's demand change sets the current record's status to "out_of_date" and fills newer_case_id with the new case's id. The evals read from these records. Each record is saved to disk after every status change, so the evals can read it after the run ends.

Demand agent — [Phase 2]

You are the Demand agent in NurseSync. Call get_demand and report how many nurses each unit needs for each shift of the week. Show the patient counts and the ratio you used so the manager can check and change them. If a day has no data or get_demand returns an error for it, list that day under "Needs manual review" and leave patient_count and nurses_needed empty in your output row. Output: a table of unit, date, shift, patient_count and nurses_needed, plus one line about any day that looks unusual.

Availability agent — [Phase 2 and Phase 3]

You are the Availability agent in NurseSync. Call get_availability for each nurse, and get_callouts for any call-outs reported since the schedule was posted. Report each nurse's available times, approved time off and call-outs. Treat approved time off and call-outs as hard limits. If a nurse's data is missing or empty, list the nurse under "Needs manual review", leave available and reason empty in your output row, and do not assume they are available. Output: for each nurse, the times they can work and the times they cannot, then the Needs manual review list.

Scheduler agent — [Phase 1]

What this agent does: Reads demand, availability and roster data, then builds a weekly draft schedule that fills every shift with an available, qualified nurse. When the Rules Checker finds violations, it fixes them and returns a revised draft. It repeats this up to 3 rounds.

Inputs it receives (in phase 1, from files; in phase 2, from the orchestrator):

- Demand rows: one row per unit/date/shift saying how many nurses are needed.
- Availability rows: one row per nurse/date/shift saying whether they can work and why not if they can't.
- Roster (from get_roster): one row per nurse with their unit, certifications and hours worked this week.
- Violations (from check_rules, with fix hints from the Rules Checker, rounds 2 and 3 only): a list of what the previous draft broke and a fix_hint for each.
- Locked assignments (call-out runs only): the rest of the week from the earlier draft. Do not change them.
- The current draft version number (1 on the first round, 2 or 3 on fix rounds).

Hard rules to follow (from 3e):

- At least 10 hours of rest between the end of one shift and the start of the next.
- No more than 40 hours per nurse per week and no more than 3 shifts in a row.
- Every shift includes at least one nurse with the required certification.
- No one is scheduled outside their availability or approved time off.

How to handle each status:

- "missing": leave that shift or nurse unscheduled. Add an open_gap row with gap_reason "missing". Never guess.
- "needs_review": use the nurses_needed or available value the row contains. Add the note to your reasoning notes and add a needs_manual_review row. Never change the number.

On fix rounds (version 2 or 3): fix each hard violation using its fix_hint field. Work by the violation's rule, nurse_id, date and shift. Fix hard violations in this order: coverage and availability first, then the rest by date and then shift, because moving a nurse can change the rest-time and hours checks. Ignore soft tradeoffs while fixing. If a fix is impossible — for example the only nurse with the required certification is on leave — do not invent a nurse. Add an unfixable row: the violation's rule, nurse_id, date, shift, and one plain sentence explaining why (e.g. "no available nurse holds the required certification for this shift"). Leave the shift as an open gap.

For a call-out (phase 3): you receive the rest of the week as locked assignments. Do not change them. Change only the affected shifts and suggest replacements with one line of reasoning for each. Put them in the draft's suggestions field.

Output: a draft in the format defined above, with the version number set to the current round number.

Example (round 1):

- Input: 24 nurses, 7 day shifts and 7 night shifts for one week, all available, no conflicts.
- Expected output: one assignment row for each nurse needed on each shift (56 rows if every shift needs 4 nurses), 0 open gaps, 0 needs_manual_review rows, short reasoning notes on any hard choice.

Rules Checker agent — [Phase 1]

What this agent does: Receives the violations that check_rules found in a draft and adds a fix_hint to each hard violation. It does not run the check, does not decide pass or fail, and does not build, defend, or modify the draft. The orchestrator compares this agent's violation list to the code's list and uses the code's list if they differ.

Inputs it receives:

- The violations that check_rules found for this draft (the orchestrator runs check_rules in code before this agent).
- The draft and the roster and rules files, for context when writing each fix_hint.

What check_rules tests:

- Rest time: at least 10 hours between the end of one shift and the start of the next, counting overnight shifts from their actual end time the next morning.
- Weekly hours: no more than 40 hours per nurse per week.
- Consecutive shifts: no more than 3 shifts in a row.
- Certification: at least one nurse with the required certification on every shift.
- Availability: no one scheduled outside their available times or on approved time off.
- Coverage: every shift has the number of nurses the demand row requires.
- Fairness (soft): the gap between the nurse with the most and the fewest night and weekend shifts is at most 2. This is reported as a soft tradeoff and does not fail a draft.

Output: the violations check_rules found, with a fix_hint added to each hard one:

- Hard violations (each with: rule, nurse_id, date, shift, found, limit, fix_hint).
- Soft tradeoffs (each with: rule, nurse_id, date, shift, note — these do not fail a draft).

A draft passes only when check_rules finds zero hard violations. The orchestrator takes pass or fail from check_rules itself, never from this agent. Soft tradeoffs are not sent back to the Scheduler. They go to the approval view only.

Example (overnight rest violation):

- Input: N003 assigned Monday night 19:00–07:00 Tuesday and also Tuesday day 07:00–19:00.
- Expected output: fail. One hard violation — rule: rest_time, nurse_id: N003, date: 2026-10-27, shift: day, found: "0 hours of rest", limit: "10 hours", fix_hint: "Remove N003 from Tuesday day shift or move Monday night to another nurse."

3c. Blast Radius

Worst-case scenario: NurseSync produces a schedule that breaks a rule or leaves a shift uncovered, and the manager approves it without noticing. Nurses on the unit are affected, but the damage can be undone: the schedule can be corrected and reissued. The blast radius stays small because every tool is read-only, nothing is sent to nurses without the manager, and the project only uses synthetic data.

Failure Modes & Safeguards

Failure mode	Worst-case impact	Safeguard
Scheduler books a nurse who is unavailable or on approved leave	A shift is uncovered and a nurse is upset	Availability rows have a hard available field; Rules Checker tests availability; manager approves before anything goes out
Draft breaks rest-time or overtime rules	Safety and compliance risk	Rules Checker loop; stop after 3 tries and hand the open violations to the manager
Demand estimate is too low	A unit is understaffed	Orchestrator recomputes from patient count and ratio, flags differences and outliers; manager sees demand numbers first
Data is missing for a nurse or a day	A nurse or shift is left out	Status "missing" leaves the row unscheduled; Scheduler lists it as a gap; never guesses
Overnight shift is dated to the wrong day, so a rest violation is missed	A nurse works with no rest between shifts	shift_start and shift_end carry full times; check_rules counts from end to start; unit-tested with Case 4
Manager approves without noticing a problem	A rule or coverage problem reaches the nurses	Approval view lists open gaps, needs_manual_review items and open violations first; nothing goes to nurses until manager approves
Call-out is reported while a draft is waiting for approval	The manager approves an out-of-date schedule	Case record is marked out_of_date; manager cannot approve it; affected shifts rerun on a new snapshot
Demand number is wrong but well-formed	A unit is understaffed	Orchestrator flags the mismatch as needs_review; manager sees it first
Tools read different data during one run, for example a call-out arrives mid-run	The draft is checked against data the agents never saw	Snapshot rule: every tool in a run reads one snapshot; a mid-run call-out starts a new run

Scheduler and Rules Checker disagree on the draft or violation format	A violation is missed, or the Scheduler cannot tell what to fix	Fixed draft and violation formats in 3b; check_rules rejects a draft it cannot parse
A flagged demand row is treated as missing	A shift is left unscheduled only because its number looked unusual	Three statuses: only "missing" leaves a shift open; "needs_review" rows are scheduled and shown to the manager
Roster nurse missing from availability	Nurse silently unscheduled	Orchestrator cross-checks nurse_id lists and adds a needs_manual_review row for any mismatch
Rules Checker agent misreports what check_rules found	A violation is hidden or invented and a broken draft is called passing	The orchestrator takes pass or fail from check_rules itself and compares the agent's violation list to the code's list; the code's list wins
check_rules result is lost, malformed or for an older draft	A broken schedule is called passing	No result means fail; the result must echo case_id and version of the draft it checked

3d. Eval Card

Case	Input	Expected output — written before you run
1 — Golden example (normal input)	24 nurses on one unit, one week, clear availability, no conflicts	Every shift is covered, 0 rule violations, no one scheduled outside availability, and a short reasoning note on any hard choice
2 — Golden example (edge case)	Two nurses on approved leave and one called out mid-week, leaving fewer nurses than the demand needs	Draft covers as many shifts as possible, lists the exact open gaps, and suggests replacement options for the manager to approve
3 — Adversarial input	Availability data is empty for one nurse, and get_demand returns an error for one day	The nurse and the day appear under "Needs manual review"; no invented data, no crash, and a clear message to the manager
4 — Edge case (overnight turnaround)	A nurse works 7 pm Monday to 7 am Tuesday, and the draft also gives her the 7 am Tuesday shift	check_rules fails the draft for rest time and names the nurse, shift and rule; the Scheduler fixes it; the final draft has 0 rest violations
5 — Loop limit reached	Hard rules that cannot all be met, for example the only nurse with a	After 3 rounds NurseSync stops, hands the manager the best draft with the open violations listed, and does not call it

	required certification is on approved leave all week	passing. The draft includes an unfixable row naming the rule, the shift and the reason the Scheduler could not resolve it.
6 — Certification and overtime breach	A draft puts a nurse without the required certification on a shift and another nurse over 40 hours	Both are reported as violations with the nurse, shift and rule, and both are fixed in the next draft
7 — Single-agent baseline	Cases 1 to 3 run through one agent that has all four tools	Record hard-rule violations and coverage for both versions; the multi-agent version has fewer violations at equal or better coverage
8 — Handoff failure	The Availability agent times out or returns nothing for the whole week	After one retry, the orchestrator stops before the Scheduler runs, tells the manager which input is missing, and builds no draft from partial data
9 — Malformed handoff	The Demand agent returns a row with a missing field or a shift value outside the format	The orchestrator rejects the result, tells the manager which field is wrong, and the Scheduler does not run
10 — Late call-out	A nurse calls out after the draft is built and before the manager approves	The draft is marked out of date, the affected shifts run again on a new snapshot with the rest of the week locked, check_rules checks the whole week, and the manager is told
11 — Demand mismatch	The Demand agent reports 4 nurses for a shift where the patient count and ratio give 5	The orchestrator flags the row as needs_review and the manager sees it first
12 — Demand outlier	The synthetic data has one deliberate outlier: Wednesday night needs 9 nurses while the other nights need 4 or 5	The orchestrator flags the row as needs_review because it is more than 2 away from the night-shift median; the draft is still built and the manager sees the flag first
13 — Misreported verdict	check_rules finds 2 hard violations and the Rules Checker agent reports only 1	The orchestrator catches the mismatch, uses the code's list of 2, and sends both back to the Scheduler
14 — Lost or stale verdict	The check_rules result is missing, unparseable, or labeled with an older draft version	The orchestrator treats it as a failure, stops, and tells the manager; the draft is never called passing

Also unit-test check_rules on its own, using hand-made schedules with known violations, because a bug there would pass bad schedules. Run the single-agent baseline on cases 1 to 3 before building the full team, because those results may change which thresholds matter.

How an eval passes: run each case that depends on the Scheduler 3 times. All 3 runs must match that case's expected output. For cases that expect a clean draft (for example 1, 4, and 6), that means the final draft has zero hard violations each run. For cases that expect a stop with open or unfixable violations (for example 5), that means all 3 runs stop after 3 rounds with the expected open violations and unfixable rows — not a passing draft. Code-only cases (8, 9, 11 and 12) run once, because code gives the same answer every time. Run the single-agent baseline (case 7) on the same 3 runs and compare the averages.

3e. Rules and Data

These numbers are assumptions for the synthetic data. Change them to match a real unit's policy. Keep the rest-time number fixed once chosen, because the overnight eval case depends on it. If the fairness gap of at most 2 cannot be met on the base case, relax it to 3.

Hard rules (a draft that breaks any of these fails)

- Every shift has the number of nurses the unit needs.
- At least 10 hours of rest between the end of one shift and the start of the next. A night shift that ends the next morning counts from its end time.
- No more than 40 hours per nurse per week and no more than 3 shifts in a row.
- Every shift includes at least one nurse with the unit's required certification.
- No one is scheduled outside their availability, on approved time off, or after a reported call-out.

Soft preferences (reported to the manager, but they do not fail a draft)

- Night and weekend shifts are spread evenly: the gap between the nurse with the most and the fewest is at most 2.
- Shift preferences are honored where possible.

Synthetic data (the Shift Board)

- Roster: nurse, unit, certifications, hours worked this week. The base case is 24 nurses on one unit, with two 12-hour shifts: day 07:00–19:00 and night 19:00–07:00. At the 40-hour limit each nurse works at most 3 shifts a week, and each shift needs 4 to 5 nurses.
- Availability: nurse, date, shift, available or not, and approved time off.
- Call-outs: nurse, date, shift, and when it was reported.
- Demand: unit, date, shift, and patient count. Nurses needed is the patient count divided by the ratio, rounded up. We assume 1 nurse per 5 patients.

Demand is a calculation in version one, not a forecast. The Demand agent shows the numbers and flags unusual days for the manager. Because it is a calculation, the orchestrator also recomputes it in code. If the recheck always matches the agent's numbers, consider replacing the Demand agent with plain code after the baseline run. The outlier threshold of 2 nurses is a starting value; adjust it after the first runs.

3f. Owners

Proposed split. Confirm or change it.

- Orchestrator code, handoff checks and status writing: Cheich Toure
- Synthetic data, snapshot and case record: Cheich Toure
- check_rules, its unit tests, and the draft and violation formats: Cheich Toure
- Demand and Availability agent prompts: Shared
- Scheduler and Rules Checker agent prompts: Kerrian Gordon
- Approval view (open gaps, needs_review items and soft tradeoffs): Kerrian Gordon
- Evals in 3d and the single-agent baseline: Kerrian Gordon

3g. Build Phases

Three phases, so this week's scope is realistic. A phase is done when its evals pass. Do not start phase 2 until phase 1 is complete.

- Phase 1 (this week): synthetic data and the snapshot, the draft and violation formats, check_rules with unit tests, the Scheduler and Rules Checker loop on one unit and one week with the demand and availability read from files, and the single-agent baseline. Evals 1, 4, 5, 6, 7, 13 and 14.
- Phase 2: the Demand and Availability agents, the orchestrator with parallel runs, retry, code checks and status writing, and the case record. Evals 3, 8, 9, 11 and 12.

- Phase 3: call-outs (get_callouts, late call-outs and out-of-date marking), the approval view, and a final run of every eval against the baseline. Evals 2 and 10.

3h. Stack and Model